MATH70119: Numerical Methods in Finance

Anandakrishnan Suresh[*], Rishi Massey[†], Alexandros Violetti[‡]

March 2026

1 Question 1: Lattice Methods

1.1 Task 1: European Option

1.1.1 a)

We price a European option with payoff $g(S_T)$ under a recombining trinomial tree using the Kamrad–Ritchken parametrisation. The interval $[0, T]$ is divided into N time steps of length

$$\Delta t = \frac{T}{N}.$$

Since the tree is recombining, after n steps the stock price at a node is determined by the net number of up minus down moves. Writing this net value as $k \in \{-n, -n+1, \dots, n\}$, the stock price at time $t_n = n\Delta t$ is

$$S_{n,k} = S_0 u^k.$$

The terminal option values are obtained from the payoff:

$$V_{N,k} = g(S_{N,k}), \qquad k = -N, -N+1, \dots, N.$$

The option price is then computed by backward induction. At each node (n, k), the value is the discounted risk-neutral expectation of the three successor nodes:

$$V_{n,k} = e^{-r\Delta t} \left(q_u V_{n+1,k+1} + q_m V_{n+1,k} + q_d V_{n+1,k-1} \right), \qquad n = N-1, \dots, 0.$$

Repeating this recursion until time 0 gives the option price

$$V_{0,0},$$

which is the time-zero price of the European option.
The pseudo-code to implement the recombining trinomial tree to price a European option is presented in Algorithm 1.

[*]CID 06042471
[†]CID 02218595
[‡]CID 06048602

Algorithm 1 Trinomial Tree Pricing for a European Option

Require: Initial stock price S_0, Kamrad–Ritchken parameter λ, number of steps N, payoff function $g(\cdot)$, volatility σ, interest rate r, maturity T

1: Check parameter constraint

$$\lambda \geq 1$$

2: Set $\Delta t = T/N$

3: Set up, middle and down factors

$$u = e^{\lambda\sigma\sqrt{\Delta t}}, \quad m = 1, \quad d = e^{-\lambda\sigma\sqrt{\Delta t}}$$

4: Compute risk-neutral probabilities

$$q_u = \frac{1}{2\lambda^2} + \frac{(r - \sigma^2/2)\sqrt{\Delta t}}{2\lambda\sigma}$$

$$q_m = 1 - \frac{1}{\lambda^2}$$

$$q_d = 1 - q_m - q_u$$

5: Compute terminal stock price vector

$$\mathbf{S}^{(N)} = (S_0 u^k)_{k=N}^{k=-N} \in \mathbb{R}^{2N+1}$$

6: Evaluate terminal payoff vector

$$\mathbf{V}^{(N)} = g(\mathbf{S}^{(N)}) \in \mathbb{R}^{2N+1}$$

7: **for** $n = N - 1$ to 0 **do**

8: Compute discounted continuation value vector

$$\mathbf{V}^{(n)} \in \mathbb{R}^{2n+1}$$

$$V_i^{(n)} = e^{-r\Delta t}\left(q_u V_i^{(n+1)} + q_m V_{i+1}^{(n+1)} + q_d V_{i+2}^{(n+1)}\right), \quad i = 0, \ldots, 2n$$

9: **end for**

10: **return** $V_0^{(0)}$

1.1.2 b)

We fix $S_0 = 100$ and $K = 100$ and compute the European call price using the trinomial tree for time steps

$$N \in \{5, 10, 20, 50, 100, 200, 500, 1000\}.$$

We repeat this for several values of the Kamrad-Ritchken parameter

$$\lambda \in \{1, 1.25, 1.5, 1.75\},$$

and compare the resulting prices with the exact Black-Scholes solution.
Figure 1 shows that for all values of λ, the trinomial price converges to the Black–Scholes price as N increases. However, for small N the choice of λ has a substantial effect on the accuracy. In

particular, the case when $\lambda = 1$ performs worst, since that implies

$$q_m = 1 - \frac{1}{\lambda^2} = 0,$$

i.e. the middle branch disappears and the scheme reduces to a binomial tree model. As observed in Figure 1, this leads to a larger discretisation error compared to values such as $\lambda = 1.25$ and $\lambda = 1.5$ which produce smoother and faster convergence. We note that the case $\lambda = 1.75$ initially underprices the option, but still converges as N becomes large.

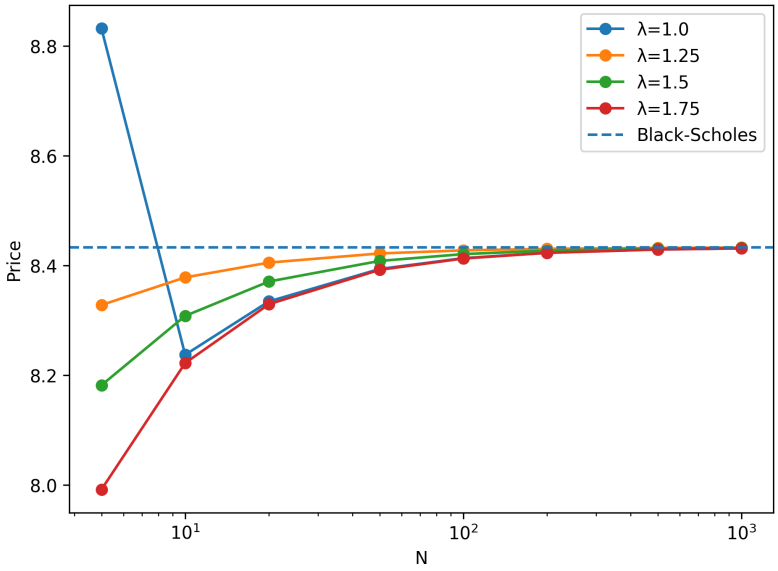


Figure 1: Trinomial European call prices against the number of time steps N for different values of λ, together with the Black-Scholes price.

We also explore the effect of changing S_0, while fixing $\lambda = 1.5$. Figure 2 shows that the same convergence pattern remains. As expected, the option price increases with S_0, since the call becomes more in the money. In additional tests with different λ, the case $\lambda = 1$ is less stable for small N, while $\lambda > 1$ (e.g. 1.25) gives smoother convergence due to the presence of a non-zero middle branch.

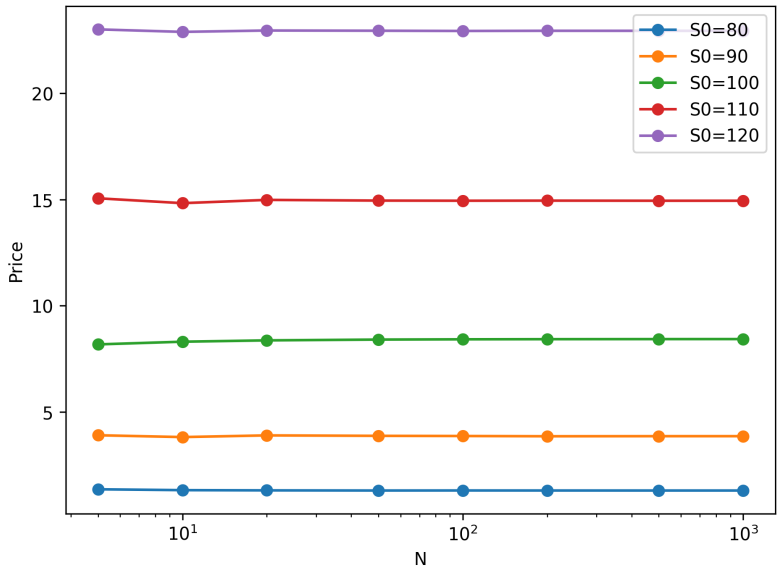


Figure 2: Trinomial European call prices against the number of time steps N for different values of S_0, with $\lambda = 1.5$.

1.1.3 c)

We now fix the number of time steps at $N = 500$ and the strike at $K = 100$ and compare the trinomial tree prices with the exact Black-Scholes prices for a European call option. We vary the initial stock price over

$$S_0 \in \{70, 80, 90, 100, 110, 120, 130\}$$

and consider the parameter values

$$\lambda \in \{1, 1.25, 1.5, 1.75\}.$$

For each pair (S_0, λ), we compute the percentage pricing error

$$\text{Error}(\%) = 100 \times \frac{|V_{\text{Tri}} - V_{\text{BS}}|}{V_{\text{BS}}}.$$

The results are reported in Table 1. Overall, the percentage errors are small for all values of λ, which indicates that for $N = 500$ the trinomial tree is already well converged. However, some values of λ are more accurate than others across the full range of S_0.
In particular, $\lambda = 1$ performs relatively poorly for lower values of S_0, with the largest error occurring at $S_0 = 70$. This is consistent with the observation from part (b), where $\lambda = 1$ was also less stable for small N. The cases $\lambda = 1.25$, $\lambda = 1.5$ and $\lambda = 1.75$ all perform better, although it is not clear which one is uniformly the best at every single value of S_0. Given Figure 1 and Table 1, we find that $\lambda = 1.5$ provides a good balance between accuracy and stability.
For the remainder of the assignment, we therefore choose to work with

$$\lambda = 1.5.$$

S_0	$\lambda = 1$	$\lambda = 1.25$	$\lambda = 1.5$	$\lambda = 1.75$
70	0.4331	0.2129	0.0129	0.0136
80	0.0819	0.0748	0.0388	0.0198
90	0.0281	0.0342	0.0090	0.0153
100	0.0471	0.0135	0.0296	0.0486
110	0.0166	0.0108	0.0058	0.0132
120	0.0004	0.0023	0.0044	0.0045
130	0.0014	0.0006	0.0016	0.0006

Table 1: Absolute percentage differences between the trinomial tree price and the Black-Scholes price for $N = 500$ and $K = 100$.

1.2 Task 2: American Option

1.2.1 a)

The algorithm is modified by allowing early exercise during the backward induction step. In the European case, the value at each node is the discounted continuation value. For an American option, we instead take the maximum between the continuation value and the immediate exercise payoff.
Hence at each node the value is

$$V_{n,k} = \max\left(e^{-r\Delta t}(q_u V_{n+1,k+1} + q_m V_{n+1,k} + q_d V_{n+1,k-1}),\, g(S_{n,k})\right).$$

This accounts for the possibility of optimal early exercise.

1.2.2 b)

Setting $K = 100$ and $N = 500$, we can see in Figure 3 that the time-zero American call price is above the intrinsic value for all values of S_0, which indicates that the option has positive time value. This difference represents the value of waiting rather than exercising immediately. Since the stock does not pay dividends, early exercise of a call option is not optimal.
Therefore, the optimal early exercise strategy at $t = 0$ is to not exercise the option and instead hold it.

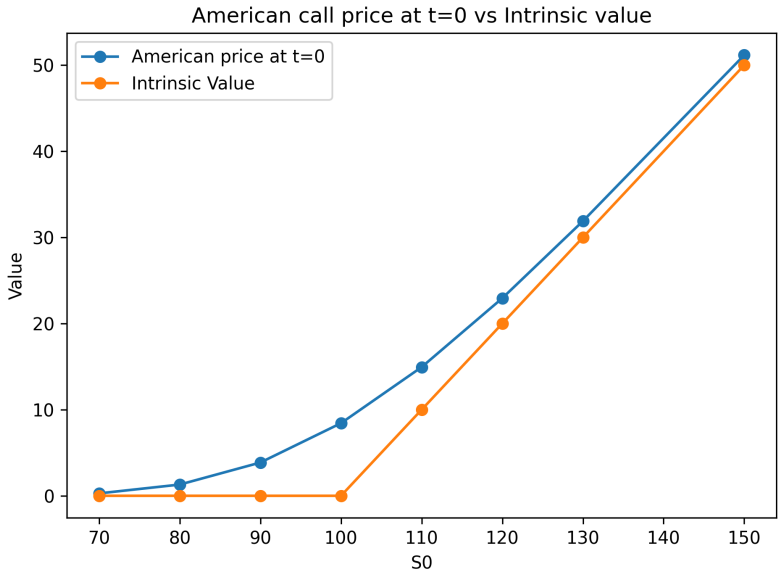


Figure 3: American call price at $t = 0$ and the intrinsic value as functions of S_0.

1.2.3 c)

We repeat part 2(b) for an American put option. As shown in Figure 4, the time zero American put price is always greater than or equal to the intrinsic value. However, for small values of S_0, the two curves are very close, which suggests that early exercise is optimal when the put is deep in-the-money. For larger S_0, the American put price is above the intrinsic value, so it is better to hold the option to take advantage of the remaining time value.
Figure 5 shows that increasing σ raises the time zero American put value and increases the gap from the intrinsic value. Higher volatility increases the probability that the option moves further in-the-money in the future, which increases the option's time value. Hence, waiting becomes more valuable and early exercise becomes less attractive.
Figure 6 shows that increasing r lowers the time zero American put value and brings it closer to the intrinsic value. A higher interest rate reduces the value of waiting, since the payoff received in the future is discounted more heavily. It also makes early exercise more attractive because the cash received from exercising can be invested at the higher rate. As a result, early exercise becomes more likely when r increases.

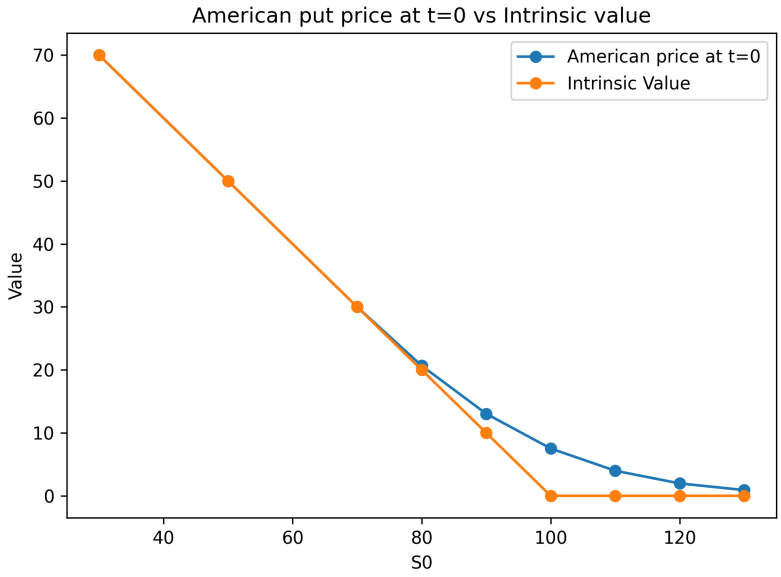


Figure 4: American put price at $t = 0$ and intrinsic value as functions of S_0.

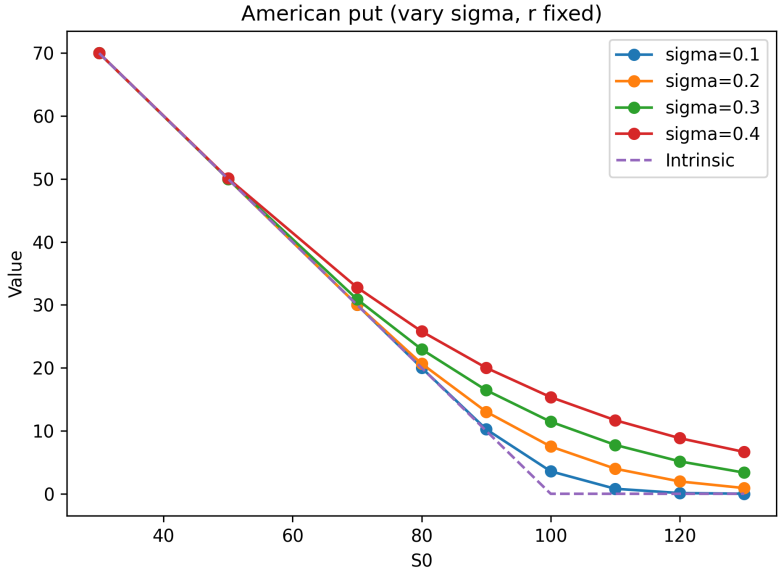


Figure 5: American put price for different values of σ, with r fixed.

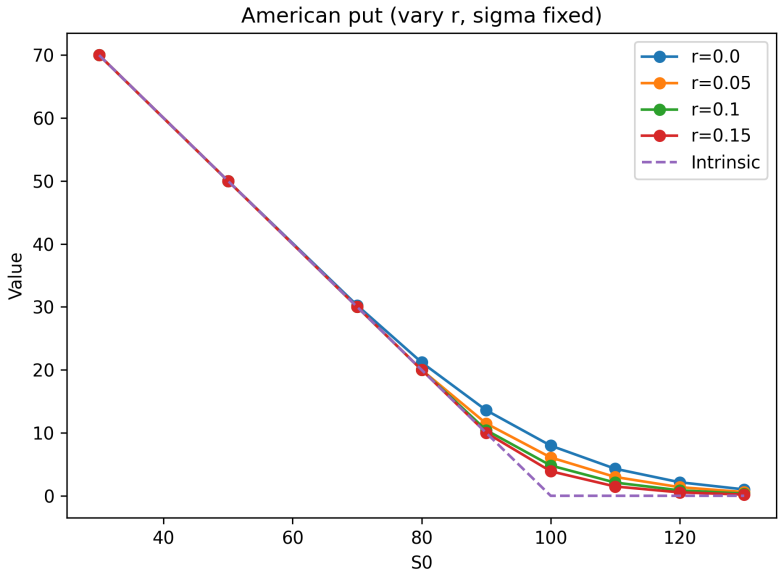


Figure 6: American put price for different values of r, with σ fixed.

1.3 Task 3: Lookback Option

1.3.1 a)

The payoff depends on the running maximum

$$M_t = \max_{0 \le u \le t} S_u,$$

so the option is path-dependent. A standard trinomial tree would not be sufficient because the current stock price does not determine the past maximum. For this reason, we extend the state space and work on a two-dimensional grid (S_n, M_n).
The stock price is written as $S_n(i) = S_0 u^i$ with $i = -n, \dots, n$. The running maximum is written as $M_n(j) = S_0 u^j$ with $j = 0, \dots, n$. Since the running maximum must be at least the current price when $i > 0$, we have the condition $j \ge \max(0, i)$. The maximum can only increase when the stock price moves up, so its evolution is deterministic once the next stock state is known. This leads to the so-called "shooting function"

$$\phi(j, i_{\text{new}}) = \max(j, i_{\text{new}}),$$

which updates the index of the running maximum.
At maturity the value matrix satisfies

$$V_{i,j}^{(N)} = M_N(j) - S_N(i), \qquad i = -N, \dots, N,\ j = \max(0, i), \dots, N,$$

$$V^{(N)} \in \mathbb{R}^{(2N+1)\times(N+1)}.$$

We then apply backward induction under the risk-neutral probabilities. At each node we compute the discounted expectation over the three branches (up, mid, down) while updating the maximum index using the shooting function. For the American case, we also compare the continuation value with the intrinsic value $M_n(j) - S_n(i)$ and take the maximum.
The pseudo-code to implement the recombining trinomial tree to price a floating strike lookback put option is presented in Algorithm 2.

Algorithm 2 Trinomial Tree Pricing for a Floating Strike Lookback Put

Require: Initial stock price S_0, Kamrad-Ritchken parameter λ, number of steps N, option type (European/American), volatility σ, interest rate r, maturity T

Check parameter constraint

$$\lambda \geq 1$$

Set $\Delta t = T/N$

Set up, middle and down factors

$$u = e^{\lambda\sigma\sqrt{\Delta t}}, \quad m = 1, \quad d = e^{-\lambda\sigma\sqrt{\Delta t}}$$

Compute risk-neutral probabilities

$$q_u = \frac{1}{2\lambda^2} + \frac{(r - \sigma^2/2)\sqrt{\Delta t}}{2\lambda\sigma}$$

$$q_m = 1 - \frac{1}{\lambda^2}, \qquad q_d = 1 - q_m - q_u$$

Define stock price states

$$S_n(i) = S_0 u^i, \qquad i = -n, \ldots, n$$

Define running maximum states

$$M_n(j) = S_0 u^j, \qquad j = 0, \ldots, n$$

Define value matrices

$$V^{(n)} \in \mathbb{R}^{(2n+1)\times(n+1)}$$

Compute terminal payoff

$$V_{i,j}^{(N)} = M_N(j) - S_N(i), \qquad i = -N, \ldots, N,\ j = \max(0, i), \ldots, N$$

Define shooting function

$$\phi(j, i_{\text{new}}) = \max(j, i_{\text{new}})$$

for $n = N - 1$ to 0 **do**
  for $i = -n$ to n **do**
    for $j = \max(0, i)$ to n **do**
      Compute continuation value

$$C = e^{-r\Delta t}\left(q_u V_{i+1,\phi(j,i+1)}^{(n+1)} + q_m V_{i,\phi(j,i)}^{(n+1)} + q_d V_{i-1,\phi(j,i-1)}^{(n+1)}\right)$$

      if European option **then**
        $V_{i,j}^{(n)} = C$
      else if American option **then**
        Compute exercise value

$$E = M_n(j) - S_n(i)$$

        $V_{i,j}^{(n)} = \max(C, E)$
      end if
    end for
  end for
end for
return $V_{0,0}^{(0)}$

1.3.2 b)

From Table 2, the prices change significantly for small N, so these are not reliable. At around $N = 50$ to 100 the prices start to stabilise but still show noticeable variation and for $N \geq 200$ the prices continue to rise but at smaller increments, indicating slow convergence towards a value near 15.7 for the European option and 15.8 for the American option by $N = 500$. However, values around $N = 200 - 250$ provide a reasonable compromise between numerical accuracy and computational cost and thus should be used for practical purposes.
The computational time increases rapidly with N because the lookback option depends on the running maximum, so the algorithm must track an additional state variable, leading to a two-dimensional state space. The number of states grows as $O(N^2)$ and the backward induction across all nodes results in an overall runtime of approximately $O(N^3)$. Therefore $N = 250$ provides a good balance between speed and accuracy.

N	European	Time (s)	American	Time (s)
5	10.253103	0.001671	10.385296	0.001546
10	11.736469	0.006727	11.868507	0.009998
25	13.263697	0.122754	13.395144	0.137714
50	14.114691	0.716884	14.246451	0.853660
100	14.750883	4.963736	14.883068	4.717948
150	15.042056	8.206947	15.174386	10.597192
200	15.218390	19.129955	15.350834	20.567748
250	15.339917	31.755890	15.472459	55.700382
300	15.430247	58.813705	15.562853	75.007035
350	15.500817	109.350090	15.633488	108.363985
400	15.557935	156.469856	15.690664	168.941450
450	15.605405	209.452636	15.738179	260.825183
500	15.645676	280.836072	15.778485	366.226685

Table 2: European and American floating strike lookback put prices together with computational times for different numbers of tree periods N.

2 Question 2: Finite Difference Methods

2.1 Task 1: Explicit, Implicit and Crank-Nicolson Scheme

2.1.1 a)

The given pricing PDE under the risk-neutral measure is

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2(t,s)s^2\frac{\partial^2 V}{\partial s^2} + rs\frac{\partial V}{\partial s} - rV = 0, \qquad t < T, \tag{1}$$

with terminal condition

$$V(T,s) = g(s). \tag{2}$$

The local volatility function is given by

$$\sigma(t,s) = \left(1 + \frac{t}{30}\right)\left(0.1 + 0.4e^{-s/50}\right). \tag{3}$$

Since the PDE is posed backward in time, we introduce the time-to-maturity variable

$$\tau = T - t.$$

Define

$$u(\tau, s) = V(T - \tau, s).$$

Using the chain rule,

$$\frac{\partial V}{\partial t} = -\frac{\partial u}{\partial \tau}.$$

Substituting into the original PDE gives

$$-\frac{\partial u}{\partial \tau} + \frac{1}{2}\sigma^2(T - \tau, s)s^2\frac{\partial^2 u}{\partial s^2} + rs\frac{\partial u}{\partial s} - ru = 0.$$

Rearranging, we have

$$\frac{\partial u}{\partial \tau} = \frac{1}{2}\sigma^2(T - \tau, s)s^2\frac{\partial^2 u}{\partial s^2} + rs\frac{\partial u}{\partial s} - ru. \tag{4}$$

When $\tau = 0$, we have $t = T$, so the terminal condition becomes

$$u(0, s) = g(s). \tag{5}$$

We therefore obtain the following initial value problem:

$$\begin{cases} \dfrac{\partial u}{\partial \tau} = \dfrac{1}{2}\sigma^2(T - \tau, s)s^2\dfrac{\partial^2 u}{\partial s^2} + rs\dfrac{\partial u}{\partial s} - ru, & \tau \in (0, T], \\ u(0, s) = g(s) & \tau = 0. \end{cases} \tag{6}$$

The time-dependent local volatility term becomes

$$\sigma(T - \tau, s) = \left(1 + \frac{T - \tau}{30}\right)\left(0.1 + 0.4e^{-s/50}\right). \tag{7}$$

2.1.2 b)

Consider the PDE obtained in part (a):

$$\frac{\partial u}{\partial \tau} = \frac{1}{2}\sigma^2(T - \tau, s)s^2\frac{\partial^2 u}{\partial s^2} + rs\frac{\partial u}{\partial s} - ru, \qquad \tau \in (0, T], \tag{8}$$

with initial condition

$$u(0, s) = g(s). \tag{9}$$

We discretise the domain $(\tau, s) \in [0, T] \times [s_{\min}, s_{\max}]$. Let

$$\Delta\tau = \frac{T}{N}, \qquad \Delta s = \frac{s_{\max} - s_{\min}}{M}.$$

Define the grid points

$$\tau_n = n\Delta\tau, \quad n = 0, 1, \ldots, N,$$

$$s_k = s_{\min} + k\Delta s, \quad k = 0, 1, \ldots, M.$$

Let

$$u_k^n \approx u(\tau_n, s_k)$$

denote the numerical approximation of the solution at grid point (τ_n, s_k).

Explicit Scheme: We let the spatial derivatives be approximated using central differences:

$$\frac{\partial u}{\partial s} \approx \frac{u_{k+1}^n - u_{k-1}^n}{2\Delta s}, \tag{10}$$

$$\frac{\partial^2 u}{\partial s^2} \approx \frac{u_{k+1}^n - 2u_k^n + u_{k-1}^n}{(\Delta s)^2}. \tag{11}$$

For the time derivative, we use a forward difference:

$$\frac{\partial u}{\partial \tau} \approx \frac{u_k^{n+1} - u_k^n}{\Delta \tau}. \tag{12}$$

Substituting these approximations into the PDE gives

$$\frac{u_k^{n+1} - u_k^n}{\Delta \tau} = a_k^n \frac{u_{k+1}^n - 2u_k^n + u_{k-1}^n}{(\Delta s)^2} + b_k^n \frac{u_{k+1}^n - u_{k-1}^n}{2\Delta s} - c_k^n u_k^n, \tag{13}$$

where

$$a_k^n = \frac{1}{2}\sigma^2(T - \tau_n, s_k)s_k^2, \qquad b_k^n = r s_k, \qquad c_k^n = r.$$

Multiplying both sides by $\Delta\tau$ and rearranging gives

$$\begin{aligned} u_k^{n+1} = &\left(\frac{\Delta\tau}{(\Delta s)^2}a_k^n - \frac{\Delta\tau}{2\Delta s}b_k^n\right)u_{k-1}^n \\ &+ \left(1 - \frac{2\Delta\tau}{(\Delta s)^2}a_k^n - \Delta\tau c_k^n\right)u_k^n \\ &+ \left(\frac{\Delta\tau}{(\Delta s)^2}a_k^n + \frac{\Delta\tau}{2\Delta s}b_k^n\right)u_{k+1}^n. \end{aligned} \tag{14}$$

Define the coefficients

$$A_k^n = \frac{\Delta\tau}{(\Delta s)^2}a_k^n - \frac{\Delta\tau}{2\Delta s}b_k^n,$$

$$B_k^n = -\frac{2\Delta\tau}{(\Delta s)^2}a_k^n - \Delta\tau c_k^n,$$

$$C_k^n = \frac{\Delta\tau}{(\Delta s)^2}a_k^n + \frac{\Delta\tau}{2\Delta s}b_k^n.$$

The explicit scheme therefore becomes

$$u_k^{n+1} = A_k^n u_{k-1}^n + (1 + B_k^n)u_k^n + C_k^n u_{k+1}^n, \qquad k = 1, \ldots, M-1. \tag{15}$$

We can obtain a much simpler and intuitive representation using matrices. Let

$$\mathbf{u}^n = \begin{bmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{M-1}^n \\ u_M^n \end{bmatrix} \in \mathbb{R}^{M+1}$$

The scheme can then be written as

$$\tilde{\mathbf{u}}^{n+1} = (I + L^n)\mathbf{u}^n, \tag{16}$$

where I is the $(M+1)\times(M+1)$ identity matrix and L^n is the tridiagonal matrix

$$L^n = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ A_1^n & B_1^n & C_1^n & & 0 \\ 0 & A_2^n & B_2^n & C_2^n & \\ \vdots & & \ddots & & \vdots \\ 0 & & A_{M-1}^n & B_{M-1}^n & C_{M-1}^n \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix} \in \mathbb{R}^{(M+1)\times(M+1)}$$

The explicit update does not determine the boundary values u_0^{n+1} and u_M^{n+1}. These are imposed using the boundary conditions

$$u(\tau, s_{\min}) = \ell(\tau), \qquad u(\tau, s_{\max}) = \lambda(\tau).$$

For a European call option,

$$\ell(\tau) = 0, \qquad \lambda(\tau) = s_{\max} - Ke^{-r\tau}.$$

After computing $\tilde{\mathbf{u}}^{n+1} = (I + L^n)\mathbf{u}^n$, the boundary entries are replaced so that

$$u_0^{n+1} = 0, \qquad u_M^{n+1} = s_{\max} - Ke^{-r\tau_{n+1}}.$$

This yields the final recursive system

$$\mathbf{u}^{n+1} = B^{n+1}\big((I + L^n)\mathbf{u}^n\big), \tag{17}$$

where $\mathbf{b} = B^n(\mathbf{u})$ is defined by $b_0 = l(\tau_n)$, $b_M = \lambda(\tau_n)$, and $b_k = u_k$ for $k = 1, \ldots, M-1$. The pseudo-code to implement the explicit scheme is presented in Algorithm 3.

Algorithm 3 Explicit Finite Difference Scheme for a European Call under Local Volatility

Require: Strike K, interest rate r, maturity T, spatial bounds $s_{\min}, s_{\max}$, grid sizes N, M

Set $\Delta\tau = T/N$

Set $\Delta s = (s_{\max} - s_{\min})/M$

Construct spatial grid

$$s_k = s_{\min} + k\Delta s, \quad k = 0, \ldots, M$$

Construct time grid

$$\tau_n = n\Delta\tau, \quad n = 0, \ldots, N$$

Define local volatility

$$\sigma(\tau, s) = \left(1 + \frac{T - \tau}{30}\right)\left(0.1 + 0.4e^{-s/50}\right)$$

Initialise payoff at maturity

$$u_k^0 = \max(s_k - K, 0)$$

for $n = 0$ to $N - 1$ **do**

Compute coefficients

$$A_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

$$B_k^n = -\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - r\Delta\tau$$

$$C_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 + \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

Construct tridiagonal matrix L^n with diagonals

$$(A_k^n,\ B_k^n,\ C_k^n)$$

Update solution

$$\tilde{\mathbf{u}}^{\,n+1} = (I + L^n)\mathbf{u}^{\,n}$$

Apply boundary conditions

$$u_0^{n+1} = 0$$

$$u_M^{n+1} = s_{\max} - Ke^{-r\tau_{n+1}}$$

Set $\mathbf{u}^{n+1} = B^n(\tilde{\mathbf{u}}^{\,n+1})$

end for

return $\mathbf{u}^N$

Implicit Scheme: An implicit scheme is obtained by approximating the time derivative $\frac{\partial u}{\partial \tau}$ using a backward difference:

$$\frac{\partial u}{\partial \tau} \approx \frac{u_k^n - u_k^{n-1}}{\Delta\tau}. \tag{18}$$

The spatial derivatives are again approximated using central differences:

$$\frac{\partial u}{\partial s} \approx \frac{u_{k+1}^n - u_{k-1}^n}{2\Delta s}, \tag{19}$$

$$\frac{\partial^2 u}{\partial s^2} \approx \frac{u_{k+1}^n - 2u_k^n + u_{k-1}^n}{(\Delta s)^2}. \tag{20}$$

Substituting these approximations into the PDE yields

$$\frac{u_k^n - u_k^{n-1}}{\Delta \tau} = a_k^n \frac{u_{k+1}^n - 2u_k^n + u_{k-1}^n}{(\Delta s)^2} + b_k^n \frac{u_{k+1}^n - u_{k-1}^n}{2\Delta s} - c_k^n u_k^n, \tag{21}$$

where

$$a_k^n = \frac{1}{2}\sigma^2(T - \tau_n, s_k)s_k^2, \qquad b_k^n = r s_k, \qquad c_k^n = r.$$

Multiplying both sides by $\Delta\tau$ and rearranging gives

$$\begin{aligned} u_k^{n-1} = &- \left(\frac{\Delta\tau}{(\Delta s)^2}a_k^n - \frac{\Delta\tau}{2\Delta s}b_k^n\right) u_{k-1}^n \\ &+ \left(1 + \frac{2\Delta\tau}{(\Delta s)^2}a_k^n + \Delta\tau c_k^n\right) u_k^n \\ &- \left(\frac{\Delta\tau}{(\Delta s)^2}a_k^n + \frac{\Delta\tau}{2\Delta s}b_k^n\right) u_{k+1}^n. \end{aligned} \tag{22}$$

Define the coefficients

$$A_k^n = \frac{\Delta\tau}{(\Delta s)^2}a_k^n - \frac{\Delta\tau}{2\Delta s}b_k^n,$$

$$B_k^n = -\frac{2\Delta\tau}{(\Delta s)^2}a_k^n - \Delta\tau c_k^n,$$

$$C_k^n = \frac{\Delta\tau}{(\Delta s)^2}a_k^n + \frac{\Delta\tau}{2\Delta s}b_k^n.$$

The implicit recursion can then be written as

$$u_k^{n-1} = -A_k^n u_{k-1}^n + (1 - B_k^n)u_k^n - C_k^n u_{k+1}^n, \qquad k = 1, \ldots, M-1. \tag{23}$$

Similar to the explicit scheme, for tractability, we use matrix representation. Let

$$\mathbf{u}^n = \begin{bmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{M-1}^n \\ u_M^n \end{bmatrix} \in \mathbb{R}^{M+1}.$$

The system can be written in matrix form as

$$(I - L^n)\tilde{\mathbf{u}}^n = \mathbf{u}^{n-1}, \tag{24}$$

where I is the $(M+1) \times (M+1)$ identity matrix and L^n is the tridiagonal matrix

$$L^n = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ A_1^n & B_1^n & C_1^n & & 0 \\ 0 & A_2^n & B_2^n & C_2^n & \\ \vdots & & \ddots & & \vdots \\ 0 & & A_{M-1}^n & B_{M-1}^n & C_{M-1}^n \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix}.$$

The system above does not incorporate the boundary conditions, so the first and last entries must be replaced after solving the linear system. Let

$$\tilde{\mathbf{u}}^n = (I - L^n)^{-1}\mathbf{u}^{n-1}.$$

The boundary values are imposed by defining

$$\mathbf{u}^n = B^n(\tilde{\mathbf{u}}^n),$$

where $B^n(\cdot)$ replaces the first and last entries. For a European call option:

$$u_0^n = 0, \qquad u_M^n = s_{\max} - Ke^{-r\tau_n}.$$

At each time step, the implicit scheme requires solving the tridiagonal linear system

$$(I - L^n)\tilde{\mathbf{u}}^n = \mathbf{u}^{n-1}.$$

Since $(I - L^n)$ is tridiagonal, this system can be solved efficiently using the Thomas algorithm. The pseudo-code to implement the Thomas algorithm and the implicit scheme is presented in Algorithm 4 and Algorithm 5 respectively.

Algorithm 4 Thomas Algorithm (TDMA) for Tridiagonal Systems

Require: Subdiagonal a, diagonal b, superdiagonal c, RHS d
$n \leftarrow$ length of d
for $i = 2$ to n **do**
    $m = a_{i-1}/b_{i-1}$
    $b_i = b_i - mc_{i-1}$
    $d_i = d_i - md_{i-1}$
end for
$x_n = d_n/b_n$
for $i = n - 1$ down to 1 **do**
    $x_i = (d_i - c_i x_{i+1})/b_i$
end for
return x

Algorithm 5 Implicit Finite Difference Scheme for a European Call

Require: Strike K, interest rate r, maturity T, grid sizes N, M, spatial bounds $s_{\min}, s_{\max}$

Set $\Delta\tau = T/N$

Set $\Delta s = (s_{\max} - s_{\min})/M$

Construct spatial grid

$$s_k = s_{\min} + k\Delta s, \quad k = 0, \ldots, M$$

Construct time grid

$$\tau_n = n\Delta\tau, \quad n = 0, \ldots, N$$

Initialise payoff at maturity

$$u_k^0 = \max(s_k - K, 0)$$

for $n = 1$ to N **do**

Compute coefficients

$$A_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

$$B_k^n = -\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - r\Delta\tau$$

$$C_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 + \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

Construct the tridiagonal system

$$-A_k^n u_{k-1}^n + (1 - B_k^n) u_k^n - C_k^n u_{k+1}^n = u_k^{n-1}$$

Define vectors for the Thomas algorithm

$$a = -A_k^n, \qquad b = 1 - B_k^n, \qquad c = -C_k^n$$

Define right-hand side

$$d = \mathbf{u}^{n-1}$$

Apply boundary conditions

$$d_0 = 0$$

$$d_M = s_{\max} - Ke^{-r\tau_n}$$

Solve the tridiagonal system

$$\mathbf{u}^n = \mathrm{TDMA}(a, b, c, d)$$

end for

return $\mathbf{u}^N$

Crank-Nicolson Scheme: The Crank–Nicolson scheme is obtained by averaging the spatial operator between the time levels $n-1$ and n. This corresponds to taking the average of the explicit and implicit discretisations.

Using the backward difference for the time derivative,

$$\frac{\partial u}{\partial \tau} \approx \frac{u_k^n - u_k^{n-1}}{\Delta\tau}, \tag{25}$$

and central differences for the spatial derivatives, we obtain

$$\frac{u_k^n - u_k^{n-1}}{\Delta\tau} = \frac{1}{2}\left[a_k^{n-1}\frac{u_{k+1}^{n-1} - 2u_k^{n-1} + u_{k-1}^{n-1}}{(\Delta s)^2} + b_k^{n-1}\frac{u_{k+1}^{n-1} - u_{k-1}^{n-1}}{2\Delta s} - c_k^{n-1}u_k^{n-1}\right] \tag{26}$$

$$+ \frac{1}{2}\left[a_k^{n}\frac{u_{k+1}^{n} - 2u_k^{n} + u_{k-1}^{n}}{(\Delta s)^2} + b_k^{n}\frac{u_{k+1}^{n} - u_{k-1}^{n}}{2\Delta s} - c_k^{n}u_k^{n}\right]. \tag{27}$$

Multiplying both sides by $\Delta\tau$ and rearranging gives

$$-\frac{1}{2}A_k^n u_{k-1}^n + \left(1 - \frac{1}{2}B_k^n\right)u_k^n - \frac{1}{2}C_k^n u_{k+1}^n = \frac{1}{2}A_k^{n-1}u_{k-1}^{n-1} + \left(1 + \frac{1}{2}B_k^{n-1}\right)u_k^{n-1} + \frac{1}{2}C_k^{n-1}u_{k+1}^{n-1}, \tag{28}$$

for $k = 1, \ldots, M-1$, where the coefficients A_k^n, B_k^n and C_k^n are the same as defined previously. Let

$$\mathbf{u}^n = \begin{bmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{M-1}^n \\ u_M^n \end{bmatrix} \in \mathbb{R}^{M+1}.$$

The Crank–Nicolson scheme can then be written in matrix form as

$$\left(I - \frac{1}{2}L^n\right)\tilde{\mathbf{u}}^n = \left(I + \frac{1}{2}L^{n-1}\right)\mathbf{u}^{n-1}, \tag{29}$$

where I is the $(M+1)\times(M+1)$ identity matrix and L^n is the tridiagonal matrix defined earlier. First, we solve the linear system

$$\tilde{\mathbf{u}}^n = \left(I - \frac{1}{2}L^n\right)^{-1}\left(I + \frac{1}{2}L^{n-1}\right)\mathbf{u}^{n-1}.$$

As in the previous schemes, the boundary values are then imposed through the boundary operator

$$\mathbf{u}^n = B^n(\tilde{\mathbf{u}}^n),$$

where, for a European call option,

$$u_0^n = 0, \qquad u_M^n = s_{\max} - Ke^{-r\tau_n}.$$

Since the matrix $\left(I - \frac{1}{2}L^n\right)$ is tridiagonal, the resulting linear system at each time step can be solved efficiently using the Thomas algorithm.

More generally, a weighted discretisation known as the θ-scheme can be written as

$$(I - \theta L^n)\tilde{\mathbf{u}}^n = (I + (1-\theta)L^{n-1})\mathbf{u}^{n-1}, \tag{30}$$

where $\theta \in [0,1]$. The cases $\theta = 0$ and $\theta = 1$ correspond to the explicit and implicit schemes respectively, while $\theta = \frac{1}{2}$ gives the Crank–Nicolson scheme. The pseudo-code to implement the Crank-Nicolson scheme is presented in Algorithm 6.

Algorithm 6 Crank–Nicolson Scheme for a European Call Option

Require: Strike K, interest rate r, maturity T, grid sizes N, M, spatial bounds $s_{\min}, s_{\max}$

Set $\Delta\tau = T/N$

Set $\Delta s = (s_{\max} - s_{\min})/M$

Construct spatial grid

$$s_k = s_{\min} + k\Delta s, \quad k = 0, \dots, M$$

Construct time grid

$$\tau_n = n\Delta\tau, \quad n = 0, \dots, N$$

Initialise payoff at maturity

$$u_k^0 = \max(s_k - K, 0)$$

for $n = 1$ to N **do**

Compute coefficients

$$A_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

$$B_k^n = -\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 - r\Delta\tau$$

$$C_k^n = \frac{1}{2}\sigma(\tau_n, s_k)^2 \frac{\Delta\tau}{(\Delta s)^2} s_k^2 + \frac{1}{2} r \frac{\Delta\tau}{\Delta s} s_k$$

Construct the right-hand side

$$\mathbf{RHS} = \left(I + \tfrac{1}{2}L^n\right) \mathbf{u}^{n-1}$$

Apply boundary conditions

$$RHS_0 = 0$$

$$RHS_M = s_{\max} - Ke^{-r\tau_n}$$

Define tridiagonal coefficients for the implicit step

$$a_k = -\tfrac{1}{2}A_k^n, \qquad b_k = 1 - \tfrac{1}{2}B_k^n, \qquad c_k = -\tfrac{1}{2}C_k^n$$

Solve the tridiagonal system

$$\left(I - \tfrac{1}{2}L^n\right) \mathbf{u}^n = \mathbf{RHS}$$

Compute solution using the Thomas algorithm

$$\mathbf{u}^n = \mathrm{TDMA}(a, b, c, \mathbf{RHS})$$

end for

return $\mathbf{u}^N$

2.1.3 c)

The European call option prices with strike $K = 100$ and maturity $T = 1$ year, computed under the local volatility model for initial stock prices in the range 80 to 120 using the explicit, implicit and Crank–Nicolson schemes, are reported in Table 3.

S_0	Explicit	Implicit	Crank–Nicolson
80	0.750455	0.751049	0.750752
85	1.474444	1.474503	1.474473
90	2.644946	2.644276	2.644611
95	4.373516	4.372207	4.372862
100	6.727497	6.725912	6.726704
105	9.712741	9.711340	9.712041
110	13.274298	13.273419	13.273858
115	17.313679	17.313419	17.313549
120	21.714282	21.714515	21.714398

Table 3: European call option prices under the local volatility model computed using different finite difference schemes.

2.2 Task 2: Implied Volatility

2.2.1 a)

The implied volatility σ_{imp} is defined as the value of σ for which the Black–Scholes price matches the observed market price C_{market}. The Black–Scholes call price is given by

$$C_{\text{BS}}(S_0, K, r, T, \sigma) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2),$$

where

$$d_1 = \frac{\ln(S_0/K) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T},$$

and $\Phi(\cdot)$ denotes the cumulative distribution function of the standard normal distribution. To compute the implied volatility, we solve the nonlinear equation

$$C_{\text{BS}}(S_0, K, r, T, \sigma) - C_{\text{market}} = 0.$$

Define the function

$$f(\sigma) = C_{\text{BS}}(S_0, K, r, T, \sigma) - C_{\text{market}}.$$

Then the implied volatility σ_{imp} is the root of f:

$$\sigma_{\text{imp}} \text{ such that } f(\sigma_{\text{imp}}) = 0.$$

Since this equation does not admit a closed-form solution for σ, a numerical root-finding method is required. In this implementation, Brent's method is used to solve

$$f(\sigma) = 0$$

on a bounded interval $[\sigma_{\min}, \sigma_{\max}]$. The pseudo-code to compute the implied volatility using `scipy.optimize.brentq` is presented in Algorithm 7 and Algorithm 8.

Algorithm 7 Black–Scholes Call Price

Require: S, K, T, r, σ

1: Compute

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}$$

2: Compute

$$d_2 = d_1 - \sigma\sqrt{T}$$

3: Compute call price

$$C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$$

4: **return** C

Algorithm 8 Implied Volatility via Brent's Method

Require: $C_{\text{market}}, S, K, T, r$

1: Define

$$f(\sigma) = C_{\text{BS}}(S, K, T, r, \sigma) - C_{\text{market}}$$

2: Set $\sigma_{\min} = 10^{-6}$, $\sigma_{\max} = 10$
3: **if** $f(\sigma_{\min}) f(\sigma_{\max}) < 0$ **then**
4: Compute σ_{imp} using Brent's method on $[\sigma_{\min}, \sigma_{\max}]$
5: **end if**
6: **return** σ_{imp}

2.2.2 b)

We fix the initial stock price $S_0 = 100$ and maturity $T = 1$ year. We use the explicit finite difference scheme to compute the European call option prices over a range of strikes $K \in [70, 130]$ under the local volatility model:

$$\sigma(T - \tau, s) = \left(1 + \frac{T - \tau}{30}\right)\left(0.1 + 0.4e^{-s/50}\right). \tag{31}$$

For each strike K, we compute the option price, C_{market}, using the numerical scheme and then calculate the corresponding implied volatility using Algorithm 8. The resulting implied volatility plot is shown in Figure 7 .

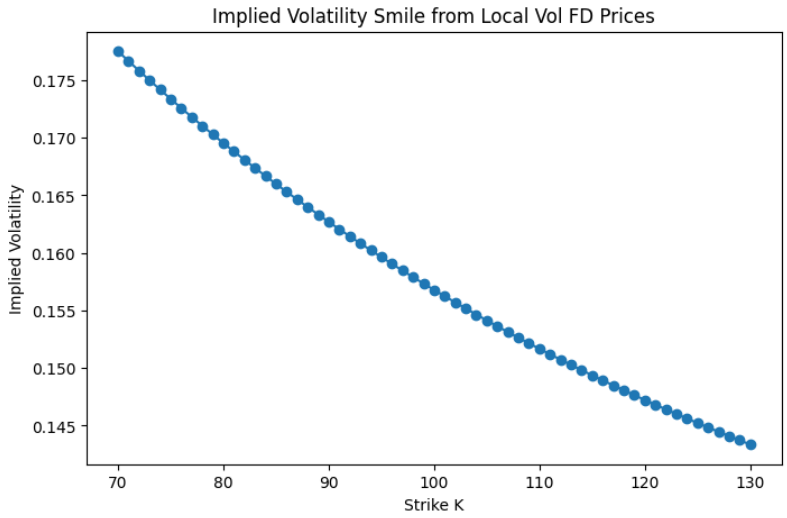


Figure 7: Implied volatility as a function of strike K, computed from option prices obtained using the local volatility finite-difference scheme.

We can observe that the implied volatility obtained under the local volatility model is not constant across strikes. Instead, the curve is downward sloping i.e., implied volatility is higher for low strike options and gradually decreases as the strike increases. This produces a volatility skew. Intuitively, this occurs because the volatility function depends on the asset price. The term $e^{-s/50}$ decreases as the stock price s increases. As a result, the local volatility is larger when the asset price is low and smaller when the asset price is high. Options with lower strikes are more sensitive to scenarios where the asset price is low, so they experience higher effective volatility. Conversely, higher strike options are more sensitive to higher asset prices where the volatility is lower, leading to smaller implied volatility. This produces the decreasing implied volatility pattern across strikes.

Now, we repeat the same procedure but replace the local volatility function by a constant volatility function. Here, we take

$$\sigma(t, s) = 15.7\% \quad \text{for all } t, s.$$

The resulting plot is shown in Figure 8.

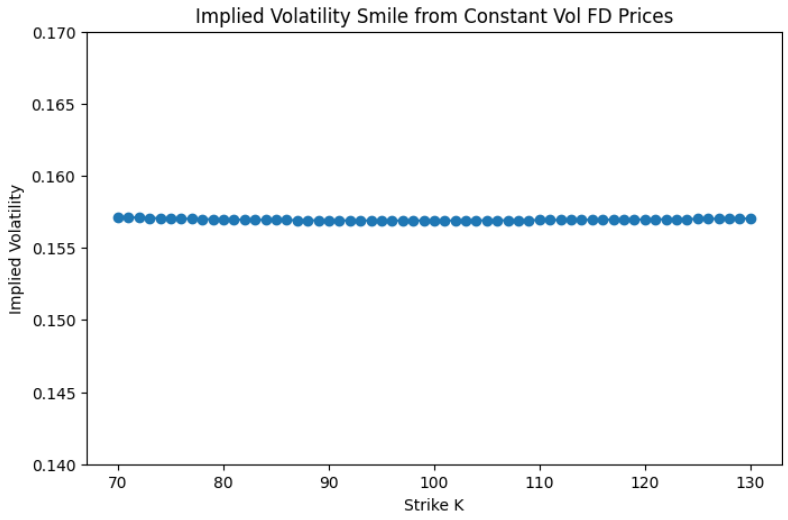


Figure 8: Implied volatility as a function of strike K, computed from option prices obtained using the constant volatility finite-difference scheme.

When the volatility is replaced by a constant function, the implied volatility is approximately constant across all strikes. The resulting implied volatility plot is therefore nearly a flat horizontal line. This occurs because the option prices are generated using the same volatility that appears in the Black–Scholes formula used to compute implied volatility. Since the pricing model and the implied volatility model are consistent, the recovered implied volatility remains equal to the constant volatility used in the model for all strikes.

Thus the key difference between the constant volatility and the local volatility is that, under the local volatility, there is a dependence on the asset price which translates into a strike-dependent implied volatility curve. On the other hand, for constant volatility, the implied volatility surface is flat. Hence, the local volatility model is able to generate volatility skews or smiles, which are commonly observed in real financial markets,

2.3 Task 3: Down-and-in Option

2.3.1 a)

Now, consider a down-and-in barrier put option with payoff

$$(K - S_T)^+ \mathbf{1}_{\{L_T \le B_{\text{in}}\}}, \qquad L_t := \inf_{0 \le u \le t} S_u.$$

A convenient way to compute the price is to use the in-out parity relationship between barrier options. For a given barrier level B_{in}, define the corresponding down-and-out barrier put with payoff

$$(K - S_T)^+ \mathbf{1}_{\{L_T > B_{\text{in}}\}}.$$

For every sample path exactly one of the events $\{L_T \le B_{\text{in}}\}$ or $\{L_T > B_{\text{in}}\}$ occurs, hence we have the identity

$$(K - S_T)^+ = (K - S_T)^+ \mathbf{1}_{\{L_T \le B_{\text{in}}\}} + (K - S_T)^+ \mathbf{1}_{\{L_T > B_{\text{in}}\}}.$$

Taking risk–neutral expectations implies the in-out parity

$$V_{\text{vanilla}}(t,s) = V_{\text{DI}}(t,s) + V_{\text{DO}}(t,s),$$

where V_{vanilla}, V_{DI}, and V_{DO} denote respectively the European put, down-and-in put, and down-and-out put prices. Therefore

$$V_{\text{DI}}(t,s) = V_{\text{vanilla}}(t,s) - V_{\text{DO}}(t,s).$$

Hence the down-and-in price can be computed once the vanilla put and the down-and-out put prices are known.

Under the local volatility model, the option value $V(t,s)$ satisfies the PDE

$$\frac{\partial V}{\partial t} + rs\frac{\partial V}{\partial s} + \frac{1}{2}\sigma^2(t,s)s^2\frac{\partial^2 V}{\partial s^2} - rV = 0, \qquad t < T,$$

with terminal condition

$$V(T,s) = (K-s)^+.$$

To solve this numerically, we truncate the domain

$$[0,T]\times[0,\infty) \quad \rightarrow \quad [0,T]\times[s_{\min}, s_{\max}],$$

and impose boundary conditions appropriate for a put option, for example

$$V(t,s_{\max}) \approx 0, \qquad V(t,s_{\min}) \approx Ke^{-r(T-t)} - s_{\min}.$$

The derivatives in the PDE are then approximated using finite differences on a grid (t_n, s_k), leading to a system of linear equations that can be solved using an implicit, explicit, or Crank–Nicolson scheme.

The down-and-out put satisfies the same PDE on the restricted domain

$$(t,s) \in [0,T]\times[B_{\text{in}}, s_{\max}],$$

with the additional absorbing boundary condition

$$V(t,B_{\text{in}}) = 0, \qquad t \le T,$$

since the option becomes worthless once the barrier is hit.
The terminal condition is

$$V(T,s) = (K-s)^+, \qquad s > B_{\text{in}},$$

and the far boundary remains

$$V(t,s_{\max}) \approx 0.$$

The PDE is again solved using a finite difference scheme on the truncated domain $[0,T]\times[B_{\text{in}}, s_{\max}]$.

After computing the vanilla put price and the down-and-out put price numerically, the down-and-in barrier put price follows from the in–out parity relation

$$V_{\text{DI}}(0,S_0) = V_{\text{vanilla}}(0,S_0) - V_{\text{DO}}(0,S_0).$$

The numerical implementation in the next section follows the implicit finite difference scheme described in Algorithm 5. The only modification is that the spatial grid is restricted to the interval $[B_{in}, s_{max}]$ and the absorbing boundary condition $V(t,B_{in}) = 0$ is imposed at each time step. The resulting tridiagonal system is solved using the Thomas algorithm described in Algorithm 4.

2.3.2 b)

We fix

$$S_0 = 100, \qquad K = 100, \qquad T = 1, \qquad r = 1\%.$$

The knock-in levels are taken to be

$$B_{\text{in}} \in \{60, 70, 80, 90\}.$$

We compute the down-and-in barrier put prices under both the local volatility model

$$\sigma(t, s) = \left(1 + \frac{t}{30}\right)\left(0.1 + 0.4e^{-s/50}\right)$$

and the constant volatility model

$$\sigma(t, s) = 15.7\% \quad \text{for all } t, s.$$

The computed vanilla put prices are

$$V_{\text{van}}^{\text{local}}(0, 100) = 5.734395, \qquad V_{\text{van}}^{\text{const}}(0, 100) = 5.740443.$$

These are very close, which is consistent with the fact that the constant volatility level 15.7% is chosen to be representative of the overall scale of the local volatility surface.
The numerical results are shown in Table 4.

B_{in}	Local vol price	Constant vol price	Percentage difference
60	0.229810	0.048038	378.39%
70	1.228333	0.707087	73.72%
80	3.508666	3.103741	13.05%
90	5.461298	5.430638	0.56%

Table 4: Down-and-in barrier put prices under the local volatility and constant volatility models. The percentage difference is computed as

$$100 \times \frac{V_{\text{DI}}^{\text{local}} - V_{\text{DI}}^{\text{const}}}{V_{\text{DI}}^{\text{const}}}.$$

Several features are immediately visible from Table 4.
First, the down-and-in barrier put price increases as the barrier level B_{in} increases. This is exactly what one would expect; when the barrier is higher, it is easier for the stock price path to hit the barrier during the life of the option, so the knock-in event becomes more likely. Hence the option is more likely to become active and therefore more valuable.
Second, as the barrier level approaches the initial stock price $S_0 = 100$, the down-and-in price becomes close to the vanilla put price. For example, when $B_{\text{in}} = 90$, we obtain

$$V_{\text{DI}}^{\text{local}}(0, 100) = 5.461298, \qquad V_{\text{DI}}^{\text{const}}(0, 100) = 5.430638,$$

which are both close to the corresponding vanilla put prices. This is natural, because when the barrier is close to the initial stock price, the event of hitting the barrier before maturity becomes highly likely, so the down-and-in option behaves similarly to the vanilla put.

Third, the local volatility model produces significantly higher prices than the constant volatility model for low barrier levels. This is especially pronounced for $B_{\text{in}} = 60$, where the percentage difference is approximately 378.39%. The reason is that the local volatility function

$$\sigma(t, s) = \left(1 + \frac{t}{30}\right)\left(0.1 + 0.4e^{-s/50}\right)$$

is larger when the stock price s is small. Hence, when the stock moves downward toward a low barrier, the instantaneous volatility increases, making barrier hitting more likely. This effect is not captured by the constant volatility model, which uses the same volatility level regardless of the stock price.

Finally, the percentage difference decreases sharply as the barrier increases. At $B_{\text{in}} = 90$, the difference between the two models is only 0.56%. This suggests that when the barrier is already close to spot, the knock-in event is likely under both models, so the stock-price dependence of the local volatility function has relatively little additional effect on the option price.

We can thus conclude that the barrier level has a strong influence on the down-and-in put price, and that the local volatility specification matters particularly for low barrier levels. In this model, the local volatility is larger when the stock price is low, which increases the likelihood of hitting a low down barrier and leads to substantially higher down-and-in option prices than under the constant volatility benchmark.

Acknowledgments

We acknowledge the use of **GPT-5** for assistance in generating code for figures and plots, as well as for LaTeX formatting of tables and results.